Following is the implementation of the application layer. Being a DTO, the *NewUser* is a simple data structure. The DTOs do not apply any business rules or logic. Hence it is OK to make the attributes public.

```
package com.glarimy.ums.app;

public class NewUser {
public String name;
public long phone;

public NewUser(String name, long phone) {
     this.name = name;
     this.phone = phone;
}

}
```

Like *NewUser*, the *UserRecord* is also a DTO. The implementation is self-explanatory:

```
package com.glarimy.ums.app;

import java.util.Date;

public class UserRecord {
public String name;
public long phone;
public Date since;

public UserRecord(String name, long phone, Date since) {
this.name = name;
this.phone = phone;
this.since = since;
}

}
```

And, finally, the *UserController* is the one that connects with the domain model to serve the user requests. It may also connect with the other infrastructure like message brokers, email servers, etc.

These kinds of classes are popularly called Controller classes largely due to the influence of the MVC pattern and Spring framework. For that matter, they can be named as you wish. However, these classes follow a simple process in serving the user requests.

Controllers map the input DTO to appropriate domain objects, invoke operations on domain objects, convert the results back into DTOs and return them to the caller.

Notice that the domain objects are never leaked beyond the application layer and the domain layer never entertains DTOs.

The *UserController* also follows the same pattern:

```
package com.glarimy.ums.app;

import com.glarimy.broker.Event;
import com.glarimy.broker.Publisher;
import com.glarimy.ums.domain.Name;
import com.glarimy.ums.domain.PhoneNumber;
import com.glarimy.ums.domain.User;
import com.glarimy.ums.domain.UserRepository;

public class UserController {
private UserRepository repo;

public UserController(UserRepository repo) {
     this.repo = repo;
}

public UserRecord add(NewUser newUser) {
Name name = new Name(newUser.name);
PhoneNumber phoneNumber = new PhoneNumber(newUser.phone);
User user = new User(name, phoneNumber);

user = repo.save(user);

Event event = new Event("", "");
Publisher publisher = new Publisher();
publisher.publish(event);

UserRecord record = new UserRecord(user.getName().getValue(),
user.getPhone().getValue(), user.getSince());

return record;
}
}
```

The *UserController* is also supposed to handle the domain exceptions, though it is ignored in the above-illustrated code. Another point that can be noticed in the code is that it is also connected to *BrokerService* for firing the events for journaling.

That's all! Any microservice of this nature consists of such domain and application layers, at a minimum. All the remaining technical stuff is handled by the infrastructure layer.

Infrastructure layer

This layer consists of databases, message brokers, directories, file servers, email servers…and the list just goes on. The infrastructure is not specific to a domain, which is obvious. It is more technical in nature and may be available as frameworks, libraries, etc, off the shelf.

In our case, *AddService* is using *BrokerService* and a set of *Framework* classes. Let's briefly explore them as well.